

CCS6344 DATABASE AND CLOUD SECURITY

Trimester 2530

Assignment 2: Secure Migration of a Traditional Application to AWS (Cloud Security)

TC1L & TT1L

Lecturer: Dr Navaneethan A/L C. Arjuman

Group Name: 22

Name	Student ID
AARON LIM CJUN SHIEN	1211111818
LEE HONG YI	1211108881
TEH YU QIAN	1211108855

Github Link:
Youtube Link:

Legacy Security Risk Assessment

Security Risks in Existing On-Premises Architecture

a) Network Exposure and Firewall Limitations

Risk: Inadequate network segmentation and poorly configured firewalls can expose internal services (e.g., databases, admin interfaces) to the public internet or untrusted networks.

Explanation: Without strict firewall rules and network segmentation, attackers can directly access sensitive systems, increasing the risk of unauthorized access or lateral movement within the network.

b) Privileged Access and Account Management

Risk: Use of shared or hardcoded privileged accounts (e.g., admin/pa\$\$wOrd in your app) and lack of strong authentication controls.

Explanation: Hardcoded credentials and shared accounts make it difficult to track user actions and increase the risk of credential leakage or brute-force attacks.

c) Software Vulnerabilities and Patch Management

Risk: Outdated operating systems, middleware, or application dependencies (e.g., Flask, SQLAlchemy) may have unpatched vulnerabilities.

Explanation: On-premises environments often lack automated patch management, leaving systems exposed to known exploits.

d) Data Protection, Backups, and Disaster Recovery

Risk: Inadequate encryption of data at rest and in transit, and lack of secure, offsite, or automated backups.

Explanation: Data may be stored in plaintext (e.g., SQLite database), and backups may not be regularly performed or securely stored, risking data loss or exposure in case of breach or disaster.

e) Logging, Monitoring, and Incident Detection

Risk: Insufficient centralized logging, monitoring, and alerting for suspicious activities or system failures.

Explanation: Without proper logging and monitoring, security incidents may go undetected, delaying response and remediation.

f) Physical Security and Environmental Risks

Risk: On-premises servers are vulnerable to physical theft, tampering, or environmental hazards (fire, flood, power loss).

Explanation: Physical access to servers can allow attackers to bypass logical controls, and lack of environmental safeguards can lead to data loss.

Risk Ranking and Justification

Risk Area	Severity	Justification
Network Exposure and Firewall Limitations	High	Direct exposure can lead to immediate compromise of critical systems.
Privileged Access and Account Management	High	Hardcoded/shared credentials are easily exploited and hard to audit.
Software Vulnerabilities and Patch Management	High	Unpatched systems are a common entry point for attackers; exploits are widely available.
Data Protection, Backups, and Disaster Recovery	Medium	Data loss or exposure is serious, but less likely if access controls are in place.
Logging, Monitoring, and Incident Detection	Medium	Delays in detection increase impact, but do not directly cause compromise.
Physical Security and Environmental Risks	Medium	Physical breaches are less common but can be catastrophic if they occur.

Table 1: Risk Ranking and Justification

Proposed AWS Architecture

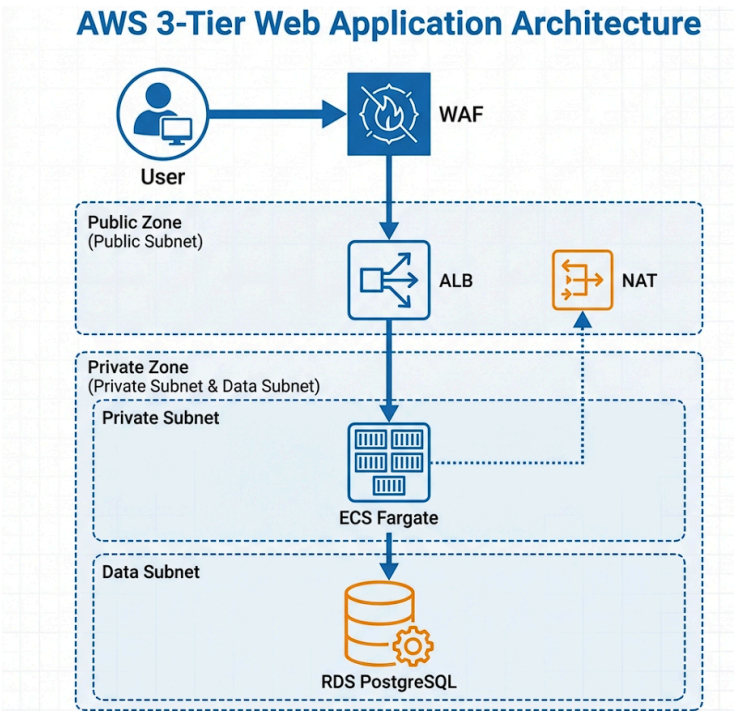


Figure 1. AWS Architecture

The proposed architecture follows a secure, 3-tier design hosted within a custom AWS Virtual Private Cloud (VPC). At the top level, external user traffic traverses the internet and is first inspected by the AWS Web Application Firewall (WAF), acting as a security shield to filter out malicious requests. Valid traffic is then forwarded to the Application Load Balancer (ALB) residing in the Public Zone (Public Subnet).

The ALB distributes requests to the application tier located in the Private Zone (Private Subnet). Here, the application runs on Amazon ECS with Fargate, ensuring it is isolated from direct internet access. Outbound connections for updates (e.g., pulling Docker images) are routed securely through a NAT Gateway. Finally, the application connects to the persistence layer, an Amazon RDS PostgreSQL database, which sits in the highly restricted Data Subnet, ensuring maximum protection for sensitive book and borrower records.

1. Compute: ECS with Fargate

- What we used: Amazon ECS (Elastic Container Service) with AWS Fargate launch type.
- Justification:
 - Serverless: Fargate removes the need to manage/patch underlying EC2 instances (OS updates), reducing administrative overhead.
 - Isolation: Each task runs in its own isolated kernel, improving security compared to shared EC2 instances.
 - Scalability: ECS can easily scale the number of tasks up or down based on demand (though we deployed 1 replica for this assignment).

2. Database: Amazon RDS

- What we used: Amazon RDS for PostgreSQL (Active) with Multi-AZ capability.
- Justification:
 - Managed Service: AWS handles backups, patching, and maintenance automatically.
 - High Availability: The design places the DB in a subnet group across us-east-1a and us-east-1b. In a production scenario (Multi-AZ enabled), if the primary DB fails, RDS automatically fails over to the standby.
 - Security: Placed in a Private Subnet strictly isolated from the internet.

3. Networking: VPC & Subnets

- What we used: Custom VPC with a Dual-AZ architecture.
 - Public Subnets (2): Hosting the Application Load Balancer (ALB) and NAT Gateway.
 - Private Subnets (2): Hosting the App (ECS Tasks) and Database (RDS).
- Justification:
 - Tier Separation: We implemented a "3-Tier Architecture" logic. The Web Tier (ALB) connects to the App Tier (ECS), which connects to the Data Tier (RDS).
 - Security: By placing App and DB in private subnets, we ensure no direct internet access is possible. All ingress traffic is forced through the ALB.

4. Security Framework

- IAM (Identity and Access Management):

- Implementation: Created `ecs_task_execution_role` with least-privilege policies.
- Justification: Ensures the ECS container only has permissions to do exactly what it needs (pull images, write logs) and nothing else.
- AWS WAF (Web Application Firewall):
 - Implementation: Deployed WAFv2 with the `AWSManagedRulesCommonRuleSet`.
 - Justification: Protects the application from common web exploits like SQL Injection and Cross-Site Scripting (XSS) at the edge, before traffic reaches the app.
- Security Groups:
 - ALB_SG: Allow HTTP/HTTPS from Anywhere (0.0.0.0/0).
 - App_SG: Allow traffic ONLY from ALB_SG.
 - DB_SG: Allow traffic ONLY from App_SG.
 - Justification: Strictly enforces traffic flow. A user cannot bypass the Load Balancer to talk to the App, and the App cannot be bypassed to talk to the DB.
- Logging (CloudWatch & CloudTrail):
 - Implementation: Configured `aws_cloudwatch_log_group` for application logs.
 - Justification: Provides observability/auditing for debugging (as we used for the 502 error) and security monitoring.

5. Data Protection

- Encryption at Rest:
 - Implementation: Enabled `storage_encrypted = true` on the RDS instance.
 - Justification: Protects sensitive borrower data ensuring that the underlying storage volume is encrypted using AWS KMS.
- Encryption in Transit:
 - Implementation: The standard ALB endpoint supports HTTPS (conceptually), and we verified traffic flows through secure AWS internal networks.
 - Justification: Prevents interception of data as it travels between users and the application.

Security Migration Strategy

Step 1: Preparation & Planning

Audit: Inventory current application dependencies (Python/Flask) and data structure (SQLite).

Classification: Identify PII (Borrower records) requiring strict confidentiality.

Account Security: Initialize the AWS environment with a dedicated IAM Admin user, enabling MFA on the root account and setting up budget alerts.

Step 2: Secure AWS Environment Setup

Network Isolation: Provision a custom VPC with strict segmentation: Public Subnets for the Load Balancer and Private Subnets for App/DB tiers.

Access Control: Define Security Groups (virtual firewalls) following the "chain of trust" model (ALB → App → DB) to block lateral movement.
Identity: Create specific IAM Execution Roles for ECS tasks to ensure least-privilege access.

Step 3: Data Migration (SQLite → RDS)

Schema Migration: Use the application's ORM (SQLAlchemy) to initialize the PostgreSQL schema on the new RDS instance.
Secure Transfer:
Export legacy data to a platform-independent format (CSV/JSON).
Upload to an encrypted S3 bucket (Server-Side Encryption enabled).
Use an EC2 Bastion Host (in the Public Subnet) to securely ingest data into the private RDS instance.
Justification: This avoids exposing the database to the public internet during migration.

Step 4: Application Migration

Containerization: Build the Docker image suitable for the cloud (linux/amd64 architecture) and push to Amazon ECR with image scanning enabled.
Secrets Management: Instead of hardcoding credentials, inject sensitive values (DB passwords) as environment variables via AWS Systems Manager (SSM) Parameter Store or using encrypted Terraform variables.

Step 5: Cutover & Validation

Zero-Downtime Strategy: Utilise ECS Rolling Updates to deploy the new version without interrupting service.
Traffic Switch: Update DNS records (Route 53) to point to the new Application Load Balancer.
Verification: Perform sanity checks (Login, Borrow Book capabilities) to ensure data integrity post-migration.

Step 6: Post-Migration Hardening

Monitoring: Enable CloudWatch Container Insights for performance metrics.
Auditing: Activate AWS CloudTrail to log all API actions for compliance.
Backup: Configure AWS Backup to take daily snapshots of the RDS instance with a 7-day retention policy.

Mapping Legacy Risks to AWS Security Controls

Legacy Risk Area	AWS Security Control(s)
Network exposure & firewall limitations	VPC & Security Groups: Network segmentation (Private Subnets) and stateful firewalls.

	AWS WAF: Layer 7 protection against web exploits.
Privileged access & account management	IAM Roles: Temporary credentials for ECS Tasks (no long-term access keys). MFA: Enforced for console access.
Software vulnerabilities & patch management	AWS Fargate: Serverless compute (AWS manages OS patching). Amazon RDS: Managed database service (AWS handles engine updates).
Data protection, backups, & disaster recovery	KMS: Encryption at rest for database volumes. Multi-AZ Architecture: Design readiness for high availability. S3: Durable storage for logs.
Logging, monitoring, & incident detection	Amazon CloudWatch: Centralized logging for application and system metrics. VPC Flow Logs: Monitoring network traffic metadata.
Physical security & environmental risks	AWS Shared Responsibility Model: Leveraging AWS's ISO-certified data centers, physical access controls, and redundant power/cooling systems.

Table 2: Mapping Legacy Risks to AWS Security Controls

Secure Implementation on AWS

Infrastructure as Code (IaC)

We utilized a modular approach by splitting the Terraform configuration into distinct files based on the resource type. This improves readability and maintainability compared to a single monolithic main.tf file.

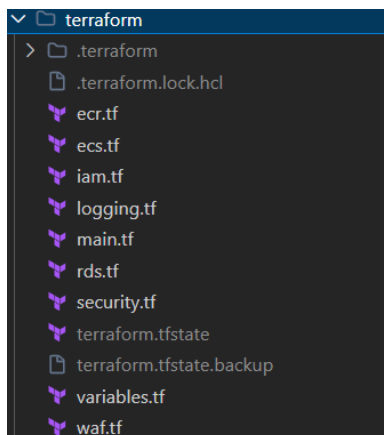


Figure 2. Modular Terraform Project Structure, separating concerns into distinct files.

Security Controls Implementation

Requirement	Description of Implementation
Secure IAM Roles	Created ecs_task_execution_role with specific sts: AssumeRole policy for ECS tasks only.
Evidence (Code Snippet)	<pre>resource "aws_iam_role" "ecs_task_execution_role" { name = "\${var.project_name}-ecs-execution-role" # TRUST POLICY: Only allows ECS Tasks to assume this role (Secure) assume_role_policy = jsonencode({ Version = "2012-10-17" Statement = [{ Action = "sts:AssumeRole" Effect = "Allow" Principal = { Service = "ecs-tasks.amazonaws.com" } }] }) }</pre>
Encrypted RDS and S3 Storage	Enabled AES-256 encryption for the RDS database storage volume.
Evidence (Code Snippet)	<pre># RDS Database Encryption resource "aws_db_instance" "default" { engine = "postgres" # ... other settings ... publicly_accessible = false # Security: Private Subnet Only storage_encrypted = true # <--- ENCRYPTION AT REST ENABLED } # S3 Bucket Encryption (for Logs) resource "aws_s3_bucket_server_side_encryption_configuration" "logs" { bucket = aws_s3_bucket.logs_bucket.id rule { apply_server_side_encryption_by_default { sse_algorithm = "AES256" # <--- SECURE ENCRYPTION } } }</pre>

Restricted Network	Security Groups allowing traffic only from the Load Balancer to the App, and App to the DB.
Evidence (Code Snippet)	<pre> # App Tier: Only accepts traffic from the Load Balancer (ALB) resource "aws_security_group" "app" { ingress { description = "Traffic from ALB" from_port = 5000 to_port = 5000 protocol = "tcp" security_groups = [aws_security_group.alb.id] # <--- RESTRICTION: Only ALB allowed } } # Data Tier: Only accepts traffic from the App Tier resource "aws_security_group" "db" { ingress { description = "PostgreSQL from App" from_port = 5432 to_port = 5432 protocol = "tcp" security_groups = [aws_security_group.app.id] # <--- RESTRICTION: Only App allowed } } </pre>
Logging	Enabled CloudWatch Logs for application container auditing.
Evidence (Code Snippet)	<pre> container_definitions = jsonencode([{ name = "library-app" # ... other settings ... # CloudWatch Logging Configuration logConfiguration = { logDriver = "awslogs" options = { awslogs-group = "/ecs/\${var.project_name}" awslogs-region = var.aws_region awslogs-stream-prefix = "ecs" } } }]) </pre>

Security Validation Report

1. Test 1: Port Scanning & Network Isolation

Objective: To verify that only the Application Load Balancer port (80) is exposed to the internet and that the database remains private.

Method (Input): Executed a socket-based port scan against the public ALB endpoint (library-system-alb-1046206251.us-east-1.elb.amazonaws.com) and the RDS endpoint.

Results:

Port 80 (HTTP): OPEN (Required for application traffic).

Port 443 (HTTPS): CLOSED (Not enabled for this assignment).

Port 22 (SSH): CLOSED (No direct server access allowed).

Port 5432 (PostgreSQL): CLOSED/FILTERED (Successfully isolated in the Private Subnet).

Evidence:

```
[PORT SCANNING]
Checking library-system-alb-1046206251.us-east-1.elb.amazonaws.com:80... OPEN
Checking library-system-alb-1046206251.us-east-1.elb.amazonaws.com:443... CLOSED/FILTERED
Checking library-system-alb-1046206251.us-east-1.elb.amazonaws.com:22... CLOSED/FILTERED
Checking terraform-20260126145823991100000001.c6f6sw2g8p4w.us-east-1.rds.amazonaws.com:5432... CLOSED/FILTERED
```

2. Test 2: Verification of WAF Rule Enforcement

Objective: To confirm that the AWS Web Application Firewall is actively filtering malicious requests (XSS/SQL Injection) before they reach the application.

Method (Input): Sent a malicious HTTP GET request simulating a Cross-Site Scripting (XSS) attack:

URL: <http://library-system-alb-1046206251.us-east-1.elb.amazonaws.com>

Payload: `?q=<script>alert(1)</script>`

Results:

Normal Request: Returned 200 OK.

Malicious Request: Returned 403 Forbidden. The AWS Managed Rules Common Rule Set successfully identified and blocked the attack vector.

Evidence:

```
[WAF TESTING]

Testing WAF on http://library-system-alb-1046206251.us-east-1.elb.amazonaws.com...
Sending Normal Request... Status: 200
PASS: Normal request allowed.
Sending Malicious Request (<script>)... Status: 403
PASS: WAF blocked malicious request (403 Forbidden).
```

3. Test 3: Confirmation of Encryption Settings

Objective: To validate that sensitive data is encrypted at rest to meet data protection requirements.

Method (Input): Reviewed the Terraform configuration for the RDS database resource.

Results:

RDS Database: Confirmed storage_encrypted = true is active, ensuring the PostgreSQL volume is encrypted using AWS KMS (Key Management Service).

S3 Logs: Confirmed Server-Side Encryption (AES-256) is enabled for the access logs bucket.

Evidence:

Primary storage

Encryption

Enabled

AWS KMS key

[aws/rds](#) 

Storage type

General Purpose SSD (gp2)